

1. O problema é usar POST para apenas buscar dados. Também não é adequado retornar todos os 12 mil alunos de uma vez, porque a resposta fica muito grande.

Redesenho: GET /api/alunos

Status: 200 OK. O ideal é usar paginação para não retornar todos os alunos de uma vez.

2. O problema é usar GET para excluir um aluno. GET deve ser usado para consulta e não para alterar dados. Também não faz sentido retornar 200 mesmo quando o aluno não existe.

Redesenho: DELETE /api/alunos/7

Status: 204 No Content se excluir e 404 Not Found se o aluno não existir.

3. O problema é retornar 200 OK e apenas "OK" depois de criar o aluno. O sistema também deveria informar qual foi o ID criado.

Redesenho: POST /api/alunos

Status: 201 Created, retornando os dados do aluno criado. Também pode usar o Location com a rota do novo aluno, por exemplo /api/alunos/10.

4. O problema é a rota ser muito grande e exigir vários IDs para chegar até uma disciplina. Isso deixa a API difícil de usar e manter.

Redesenho: GET /api/disciplinas/12

Status: 200 OK se encontrar e 404 Not Found se a disciplina não existir.

5. O problema é retornar 200 OK mesmo quando ocorreu um erro. Também está retornando HTML em uma API que deveria responder em JSON.

Redesenho: GET /api/alunos/7/matriculas

Status: 200 OK se encontrar as matrículas e 404 Not Found se o aluno não existir. O erro também deve ser retornado em JSON.

6. O problema é usar PUT para alterar somente uma parte dos dados. Como apenas a nota parcial será alterada, o PATCH é mais adequado. Também não é ideal colocar os dados da alteração na query string.

Redesenho: PATCH /api/alunos/7/disciplinas/12

Body: { "notaParcial": 8.5 }

Status: 200 OK ou 204 No Content. Se o aluno ou disciplina não existir, 404 Not Found.

Desafio:

A API precisa ter versionamento. Assim é possível criar uma nova versão sem quebrar o aplicativo antigo.

As rotas poderiam ficar assim:

/api/v1/alunos

/api/v2/alunos

O aplicativo antigo continuaria usando a versão v1 e o novo poderia usar a v2.